## Overview

Clean code and modular design make AI systems easier to maintain, debug, and scale. Without them, model and data code becomes brittle and hard to reuse across experiments and production jobs.

## Clean Code Principles

- Use clear names for variables and functions.
- Keep functions focused on one responsibility.
- Avoid duplication by extracting shared logic.
- Write readable control flow and keep side effects explicit.

## Modular Design

Split functionality into modules/packages by responsibility (data loading, feature transformation, training, evaluation). Define clear interfaces between components so each part can evolve independently.

## Documentation Practices

- Add docstrings for public functions and classes.
- Keep README files updated with setup, usage, and assumptions.
- Use inline comments sparingly for non-obvious logic.
- Maintain high-level architecture notes for handoff and onboarding.

## AI-Specific Examples

- Separate one-off training experiments from reusable utility modules.
- Build reusable data loaders and feature transformer functions instead of copy-pasting notebook cells.

## Common Pitfalls

- Monolithic notebooks that combine ingestion, training, evaluation, and deployment logic in one place.
- Hardcoded paths, environment assumptions, or secrets in code.

## Interview Prep Checklist

- Refactor a script into small reusable modules.
- Explain how modular design improves testing and deployment reliability.
- Explain why documentation quality matters for team handoffs and incident response.